THREAT HUNTING REPORT

njRAT Trojanized CrossFire Game Campaign

Malware Family: njRAT Variant | Classification: Remote Access Trojan (RAT)

TLP: AMBER — For Authorized Distribution Only

Report Date	:	June 2026
Analysis Type	:	Static + Dynamic Malware Analysis
Sample	:	CTF_Loader.exe / Tools.exe
SHA-256	:	c183d695fa778f9ba71265dbd01cf01936c69ccb4ad0e5c23cad4b242dfca0e9
Environment	:	Windows 10 Flare-VM (Oracle VirtualBox)
C2	:	51.77.192.109:6522
Status	:	C2 Dead — Simulated via patched binary + Python socket server

1. Executive Summary

This report documents the threat hunting investigation of a malicious binary distributed as a trojanized CrossFire game installer shared over Discord. The sample is a fully functional njRAT Remote Access Trojan (RAT) implant compiled in .NET/VB, configured to communicate with a Command & Control (C2) server over a custom TCP protocol on port 6522.

During initial triage, a process named “CTF Loader.exe” was identified consuming abnormal system resources. Although the process name resembled the legitimate Windows CTF Loader component (ctfmon.exe/CTF Loader), further inspection revealed several inconsistencies.

The executable was running from a non-standard path rather than a trusted Windows system directory, and the process was executing under the WoW64 subsystem on a native x64 host. Comparison with the legitimate Windows component showed that the observed process architecture and execution context did not match the expected behavior of the genuine system binary.

Upon extraction and controlled analysis, the binary was found to implement a broad set of malicious capabilities including: host fingerprinting, keylogging with registry-based caching, desktop screenshot capture, arbitrary plugin loading, persistence via registry run keys and the Startup folder, sandbox evasion via process blacklist checks, and firewall rule manipulation. All Collected data is exfiltrated to the attacker's C2 over a custom delimiter-based TCP protocol.

⚠ THREAT LEVEL: HIGH

The implant provides the attacker with full remote control of the victim machine, credential harvesting capability via keylogging, and real-time surveillance via screenshot capture and active-window beaconing.

2. Detection Timeline & Initial Triage

2.1 How the Infection Was Detected

The infection was surfaced through the following sequence of user-reported and analyst-observed events:

- Victim downloaded what they believed to be a CrossFire game binary from a Discord server.
- Immediately after execution, the victim reported significant, unexplained system slowdown.
- A process named "CTF Loader" was identified consuming abnormally high CPU and memory resources.
- The process was found to be executing under the WoW64 subsystem on a native 64-bit host.
- The host was immediately isolated from the network to prevent further data exfiltration.
- The malware binary was securely extracted and transferred to a Flare-VM analysis environment.

2.2 Detection Summary

Stage 1 — Initial Infection : Victim ran trojanized game installer obtained via Discord
Stage 2 — Suspicious Activity: Abnormal CPU usage + WoW64 process named after a x64 system process anomaly detected
Stage 3 — Incident Response : Host isolated, sample extracted, analysis initiated

3. Malware Overview

3.1 Sample Identification

Type	Value
Family	njRAT Variant
Type	Remote Access Trojan (RAT)
SHA-256	c183d695fa778f9ba71265dbd01cf01936c69ccb4ad0e5c23cad4b242dfca0e9
SHA-1	3ec1e5e14b33b3f389e1e192659764e2f4368de4
MD5	63db40196691060c8b9ec7d7c2292ff3
Filenames	"CTF Loader.exe" | Tools.exe | 07cc4935e9c1b299f68707836962cbd4.exe
Mutex	07cc4935e9c1b299f68707836962cbd4
C2 Server	51.77.192.109:6522
Delimiter	Y262SUCZ4UJJ

3.2 Infection Chain Overview

The diagram below illustrates the end-to-end infection chain from initial delivery to C2 communication:

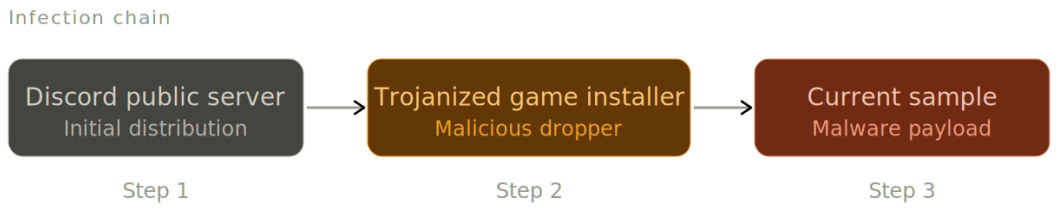

Figure 1 — Infection chain from Discord delivery to C2 beaconing

3.3 Execution Flow

The following diagram details the internal execution flow of the implant, including initialization, evasion checks, persistence setup, and C2 command dispatch:

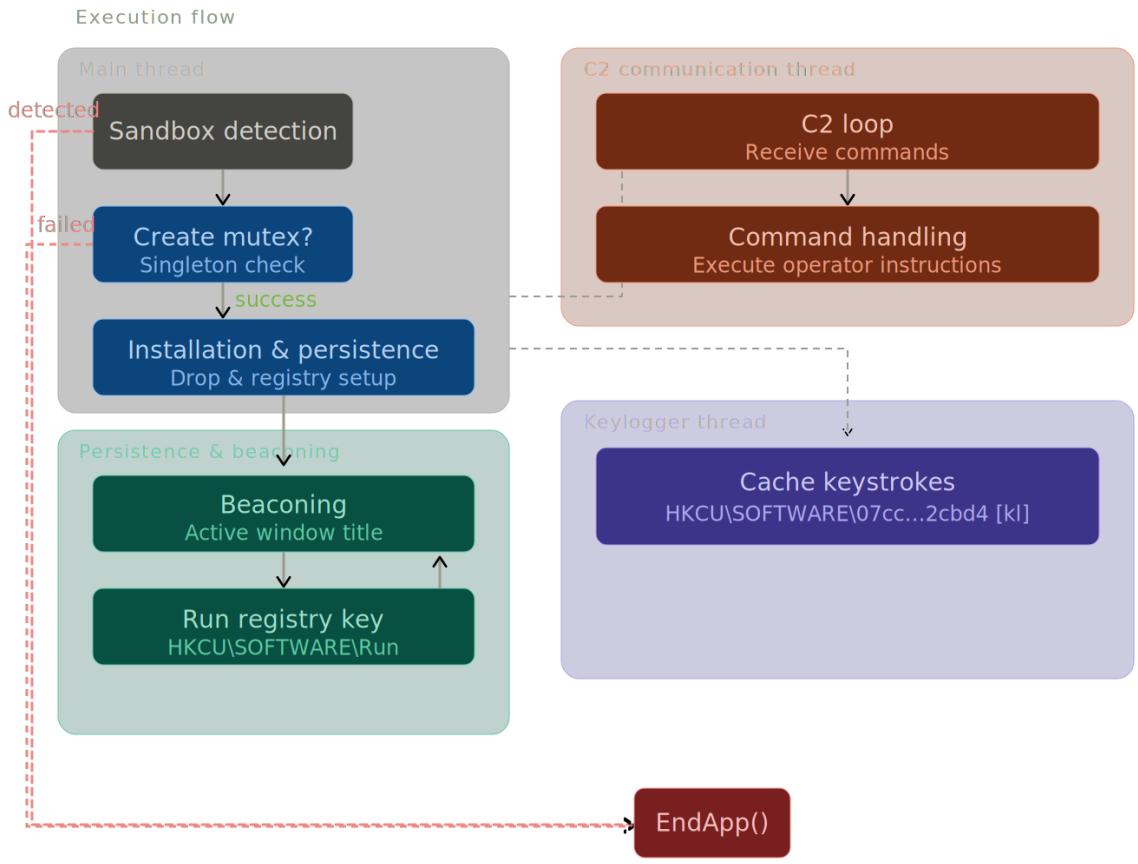

Figure 2 — Full execution flow of the njRAT implant

4. Persistence & Installation

4.1 File Drops

Upon execution, the malware copies itself to the following locations to ensure survivability across reboots and user sessions:

- All accessible drive partitions → Tools.exe
- %TEMP%\CTF Loader.exe
- %APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\07cc4935e9c1b299f68707836962cbd4.exe

4.2 Mutex Creation

The malware creates a named mutex to prevent duplicate infections on the same host:

```
Mutex Name: 07cc4935e9c1b299f68707836962cbd4
```

4.3 Zone Identifier Bypass

To suppress Windows' 'Open File – Security Warning' dialog for files downloaded from the internet or network shares, the malware sets the following environment variable at the user scope:

```
Environment.SetEnvironmentVariable("SEE_MASK_NOZONECHECKS", "1",
EnvironmentVariableTarget.User)
```

4.4 Firewall Rule Creation

The malware programmatically adds an inbound firewall allow-rule for its own binary using netsh.exe, bypassing Windows Firewall for its C2 traffic:

```
Interaction.Shell("netsh firewall add allowedprogram \"" + Config.CurrentPath.FullName +
"\" \"" + Config.CurrentPath.Name + "\" ENABLE")
```

4.5 Registry Run Key Persistence

A registry value is written to the HKCU Run key to launch the implant at every user logon:

```
Key   : HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
Value : 07cc4935e9c1b299f68707836962cbd4
Data  : "%TEMP%\CTF Loader.exe .."
```

4.6 Startup Folder Persistence

An additional persistence mechanism places the binary directly in the Startup folder, ensuring execution regardless of registry access:

```
Path: %APPDATA%\Microsoft\Windows\Start
Menu\Programs\Startup\07cc4935e9c1b299f68707836962cbd4.exe
```

4.7 Evidence Screenshots

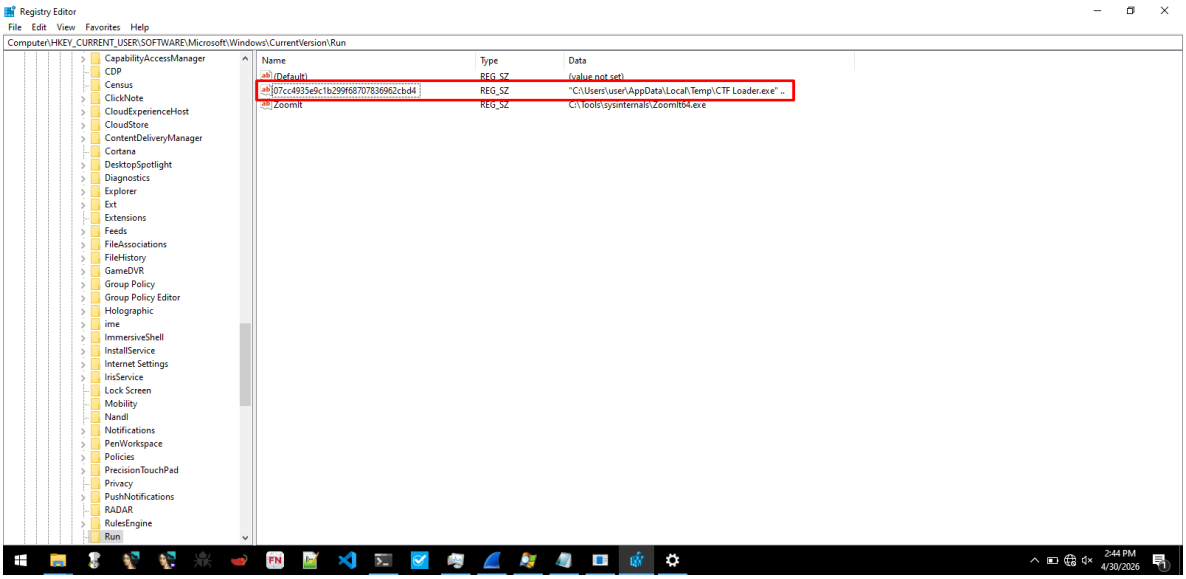

Figure 3 — Registry Run key created by the implant

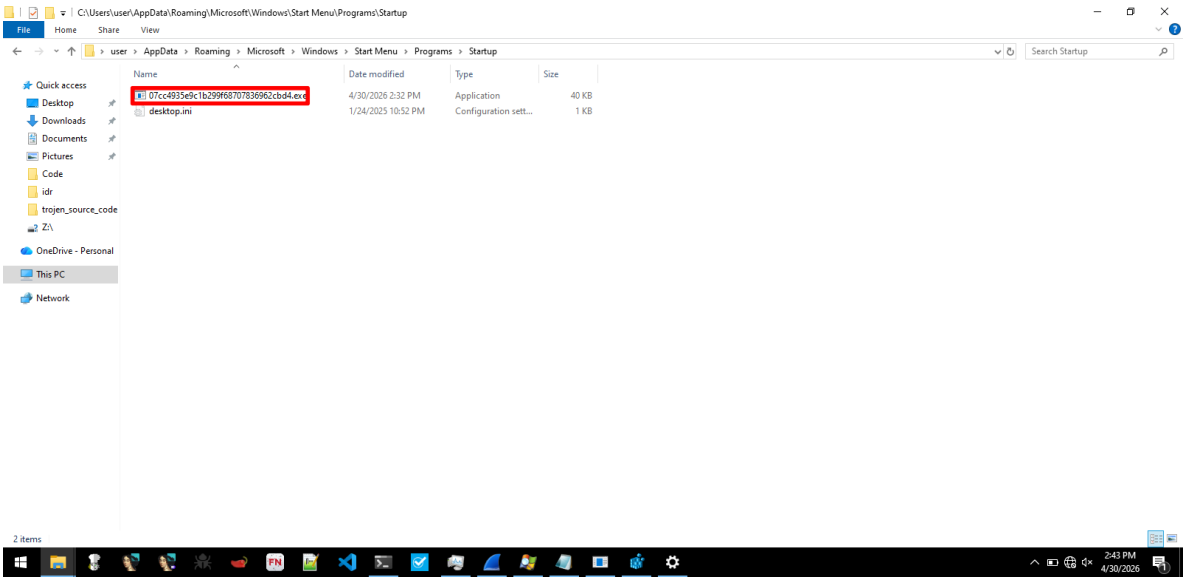

Figure 4 — Startup folder drop confirming persistence

5. Sandbox & Analysis Evasion

5.1 Process Blacklist Check

At startup, the implant enumerates all running processes and compares them against a hardcoded blacklist of 19 security analysis, debugging, and monitoring tools. If a matching process is detected, the malware immediately terminates without executing its malicious functionality. This behavior is consistent with MITRE ATT&CK T1497 — Virtualization/Sandbox Evasion.

To enable dynamic analysis within the lab environment, the blacklist entries were patched in the binary prior to execution. Specifically, the string "Not " was prepended to each blacklisted process name, preventing legitimate analysis tools from matching the malware's comparison logic while preserving the original code path. This modification effectively bypassed the self-termination mechanism and allowed the sample to execute normally for behavioral observation and C2 protocol analysis.

The following screenshots show the **dnSpy** decompiled code sections responsible for the process enumeration, blacklist comparison, and kill-switch logic, as well as the modifications applied to bypass the anti-analysis checks.
:

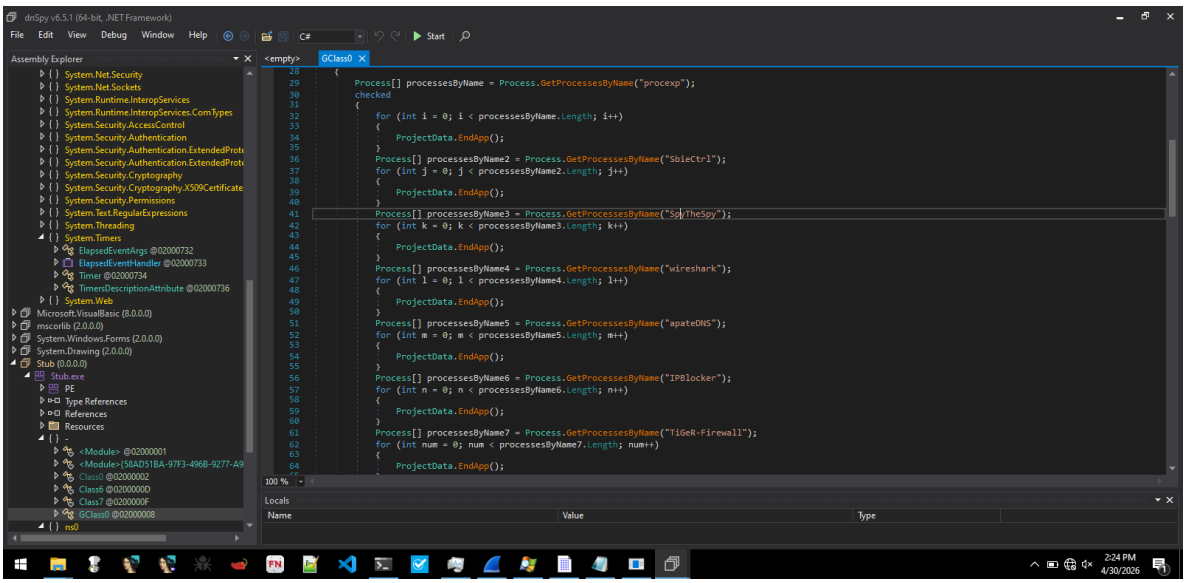

Figure 5 — Sandbox evasion: process enumeration and blacklist comparison (part 1)

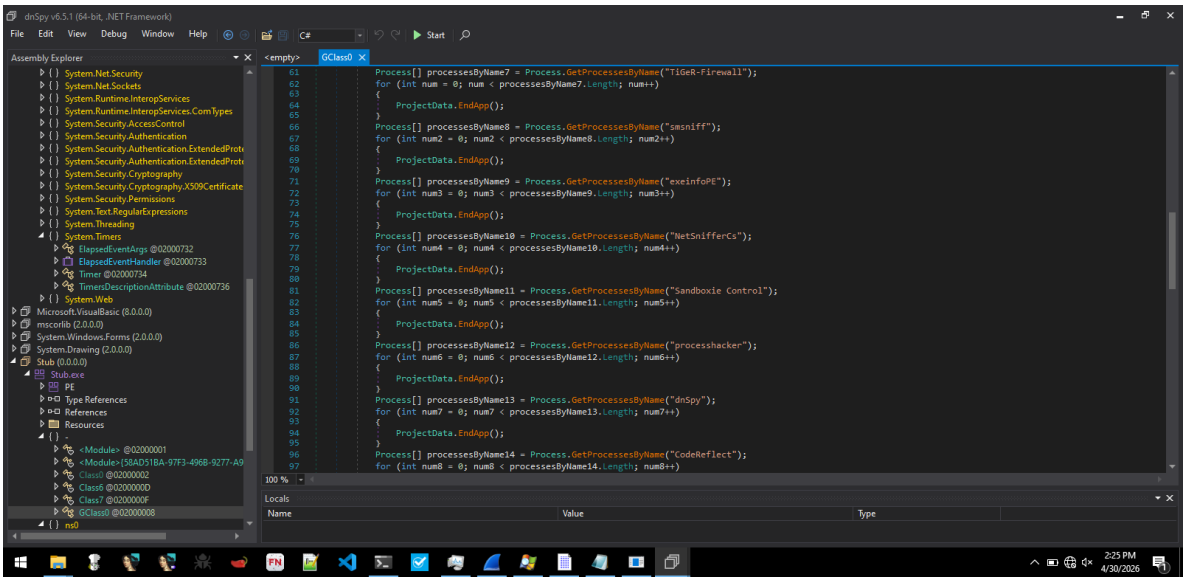

Figure 6 — Sandbox evasion: process enumeration and blacklist comparison (part 2)

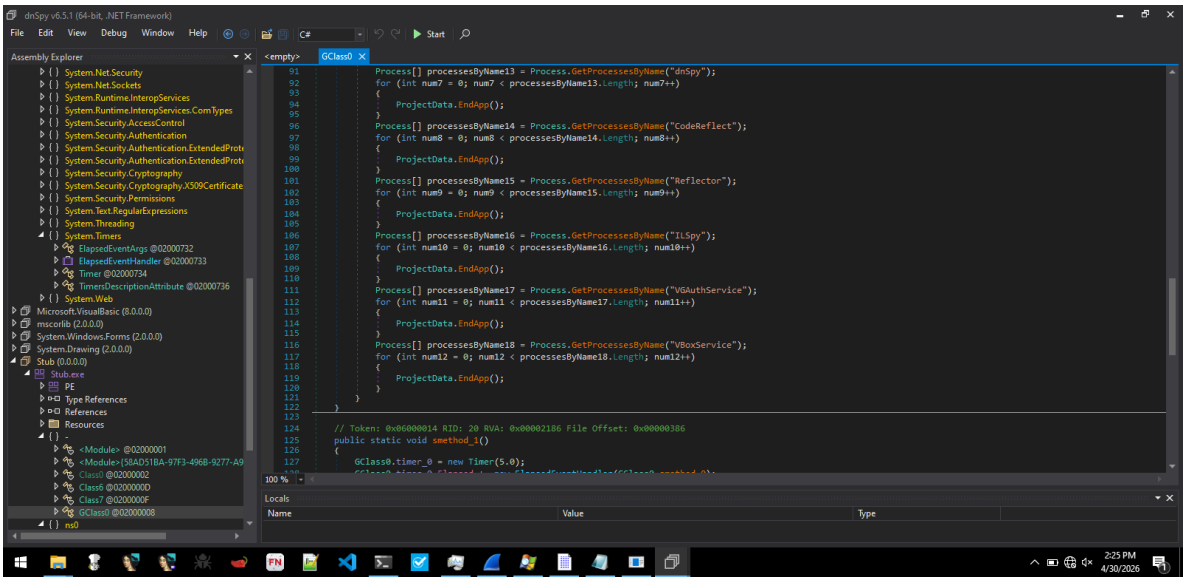

Figure 7 — Sandbox evasion: termination logic upon blacklisted process detection

6. Host Fingerprinting

6.1 Fingerprint Structure

Upon successful initialization and C2 connection, the malware transmits a two-part structured fingerprint of the victim machine. Each field after “ll” is Base64-encoded and delimited by the string Y262SUCZ4UJJ.

Part 1 — Bot Identification

```
<len>ll
<bot uuid>
<pc name>
<user name>
<current date>
<OS version>
<webcam present>
<AV product 1>
<AV product 2>
<AV product 3>
```

Part 2 — Configuration Beacon

```
<len>inf
<Config.ServerIP>:<Config.port>
<Config.TEMP>
<Config.CTF_Loader>
<COPY_TO_TMP>  <COPY_TO_STARTUP>  <USE_RUN_REGKEY>  <IsCriticalProcessProtectionEnabled>
```

6.2 Decoded Fingerprint Example

The following shows a real decoded fingerprint captured during analysis of the live implant:

```
Bot UUID    :  pharaoh player_1AED6D17
Hostname    :  DESKTOP-1BQJ37A
Username    :  user
Date        :  26-04-30
OS          :  Win 10 Pro SP0 x64
Webcam      :  No
AV          :  Windows Defender (x3)
C2 Target   :  51.77.192.109:6522
Copy Temp   :  True  |  Copy Startup: True  |  Use RegKey: True  |  Critical Process: True
```

7. Keylogging

7.1 Mechanism

The malware hooks the keyboard at the system level and caches all captured keystrokes into the Windows registry. This approach allows the malware to persist keylog data across sessions without writing to disk in a way that would trigger file-based AV scanning:

```
Key : HKEY_CURRENT_USER\SOFTWARE\07cc4935e9c1b299f68707836962cbd4
Value Name : [kl]
```

7.2 C2 Retrieval

The attacker retrieves the cached keylog buffer by issuing the 'kl' command over the C2 channel. The malware reads the [kl] registry value and transmits its content back to the C2 as Base64-encoded data.

7.3 Evidence

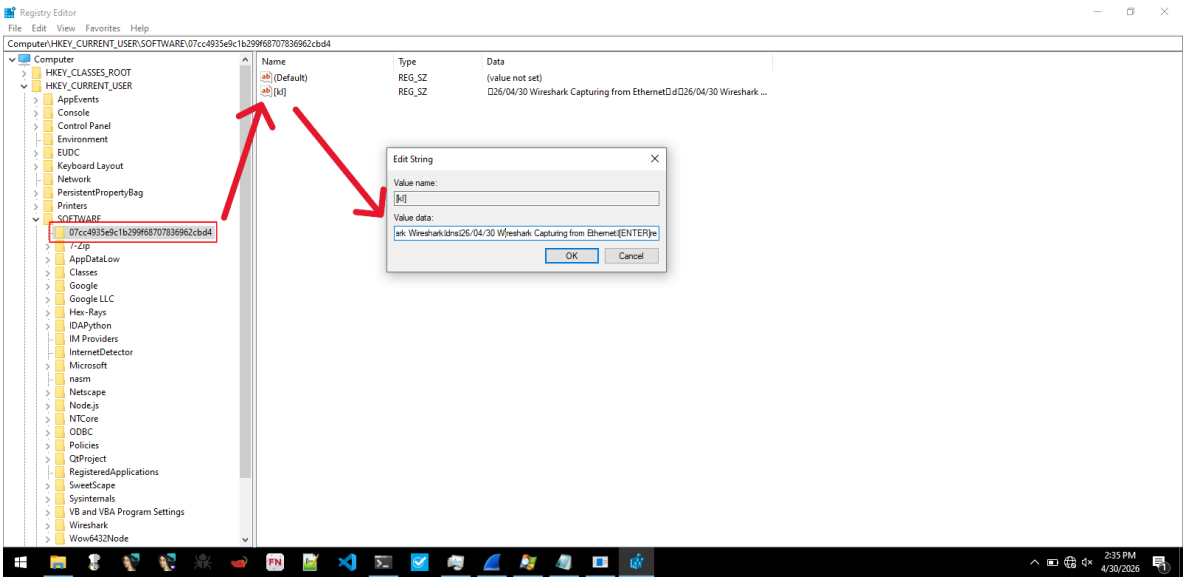

Figure 8 — Registry-based keylog cache: HKCU\SOFTWARE\<mutex>\[kl]

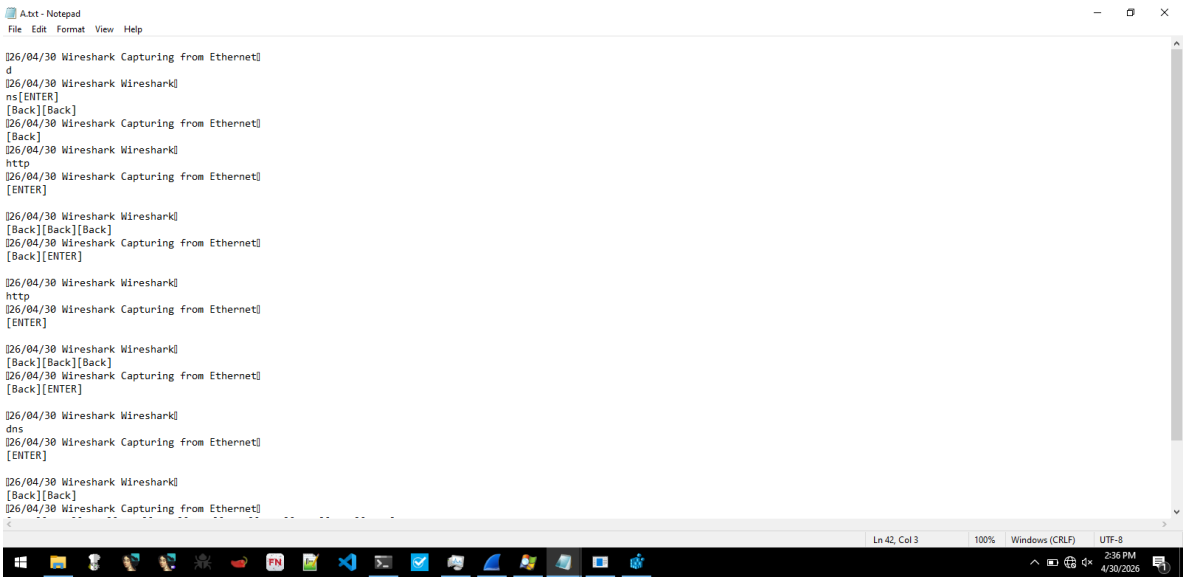

Figure 9 — Dumped keystrokes retrieved from registry during analysis

8. Network Communication & C2 Protocol

8.1 C2 Discovery

Initial network analysis was performed using **FakeNet-NG**, which intercepted outbound TCP connections from the implant to 51.77.192.109 on port 6522. The captured traffic confirmed the use of Base64 encoding and revealed the custom delimiter-based protocol format.

Since the real C2 was confirmed dead at the time of analysis, the hardcoded C2 IP in the binary was patched to redirect to a local Python socket server at 192.168.1.12:6522. This allowed full simulation of the C2 protocol and dynamic capability testing.

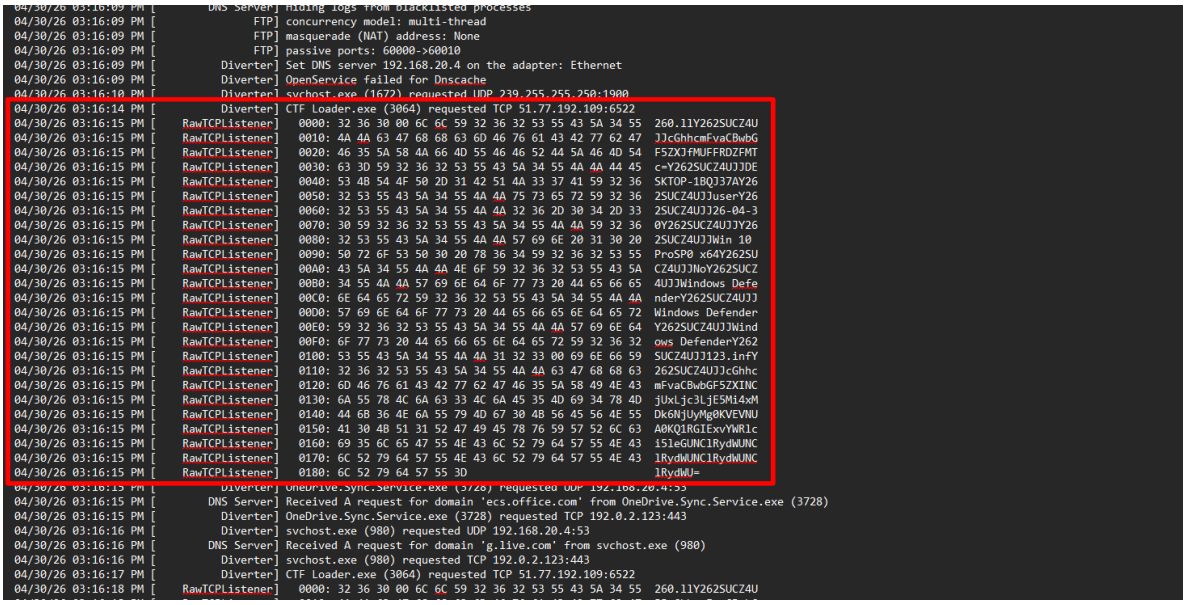

Figure 10 — FakeNet-NG capture showing outbound C2 connection attempt to 51.77.192.109:6522

8.2 Protocol Format

All messages exchanged between the implant and **C2** follow this structure:

```
"<len_in_ascii>\0<command>Y262SUCZ4UJJ<arg1>Y262SUCZ4UJJ<arg2>..."
```

Delimiter : Y262SUCZ4UJJ (used as field separator in all messages)
Encoding : Base64
Transport : Raw TCP, port 6522

8.3 Supported C2 Commands

8.3.1 Received Commands (C2 → RAT):

Type	Value
ll	Re-send host fingerprint by resetting the connection
kl	Retrieve cached keylog buffer from registry [kl] value
prof	Registry profile operations: `~` set value, `!` set and echo value, `@` delete value
rn	Download and execute a file (by URL or inline gzip) with a given extension
inv	Invoke a named method on a loaded plugin object
ret	Retrieve return value of a plugin's GT() method
CAP	Capture desktop screenshot at specified resolution (width x height)
un	Uninstall/control: `~` full uninstall, `!` terminate, `@` restart
up	Update the RAT: replace its binary with a new one from C2
Ex	Execute the ind() function of a loaded plugin
PLG	Load arbitrary .NET blob into memory as a plugin object

8.3.2 Commands Response (RAT → C2):

Type	Value
ll	Initial beacon/registration with base64 encoded system info
inf	Sends base64 encoded configuration info
act	Reports currently active foreground window
kl	Sends base64 encoded keylogger buffer in response to “kl” command
CAP	Screenshot response (Row PNG bytes)
pl	Plugin load acknowledgment (0 = success, 1 = not found)
ret	Returns result from a plugin's GT() method
PLG	Requests plugin from server when `Ex` is received but no plugin loaded
MSG	Status/error messages (e.g. "Update ERROR", "Executed As...")
bla	Acknowledgment/ready signal (sent after “up” and “rn” commands)
ER	Error reporting with the command name and exception message

9. C2 Simulation & Capability Validation

9.1 Lab Setup

Because the original Command and Control (**C2**) server (51.77.192.109:6522) was no longer operational at the time of analysis, a controlled laboratory environment was created to simulate the malware's network

communications and validate its functionality. To achieve this, the hardcoded **C2** IP address within the binary was patched and redirected to a locally hosted server at 192.168.1.12:6522.

The objective of the simulation was not to recreate the full attacker infrastructure, but rather to demonstrate and verify selected malware capabilities in a safe environment. During testing, the following protocol messages and commands were successfully simulated, captured, and analyzed:

- ✓ **II** — Host registration and fingerprint transmission
- ✓ **inf** — To receive configuration info
- ✓ **act** — To receive current active window name
- ✓ **kl** — Retrieval of cached keylog data from the registry
- ✓ **CAP** — Desktop screenshot capture and exfiltration
- ✓ **un** — Malware uninstall functionality

A lightweight custom C2 application was developed specifically for this analysis. The backend was implemented in Python and handled socket communications, command transmission, and response processing. The operator interface was developed using HTML, Vanilla JavaScript, and Bootstrap to provide a simple web-based management panel for interacting with the implant and visualizing returned data.

Network traffic generated during the simulation was analyzed exclusively using Wireshark. All packet captures were filtered using the following display filter to isolate communications between the implant and the simulated C2 server:

```
ip.addr == 192.168.1.12
```

This approach allowed direct observation of command transmission, beacon traffic, Base64-encoded payloads, and returned screenshot/keylog data without requiring access to the original attacker infrastructure.

The complete source code for the simulated C2 environment, including both the Python backend and web-based frontend, will be made available in the accompanying GitHub repository alongside this report to support reproducibility and independent validation of the analysis.

9.2 Simulated C2 Session — Active Window Beaconing

The implant beacons the currently active window title to the C2 in real time. The following captures show decoded beacon packets during the simulation:

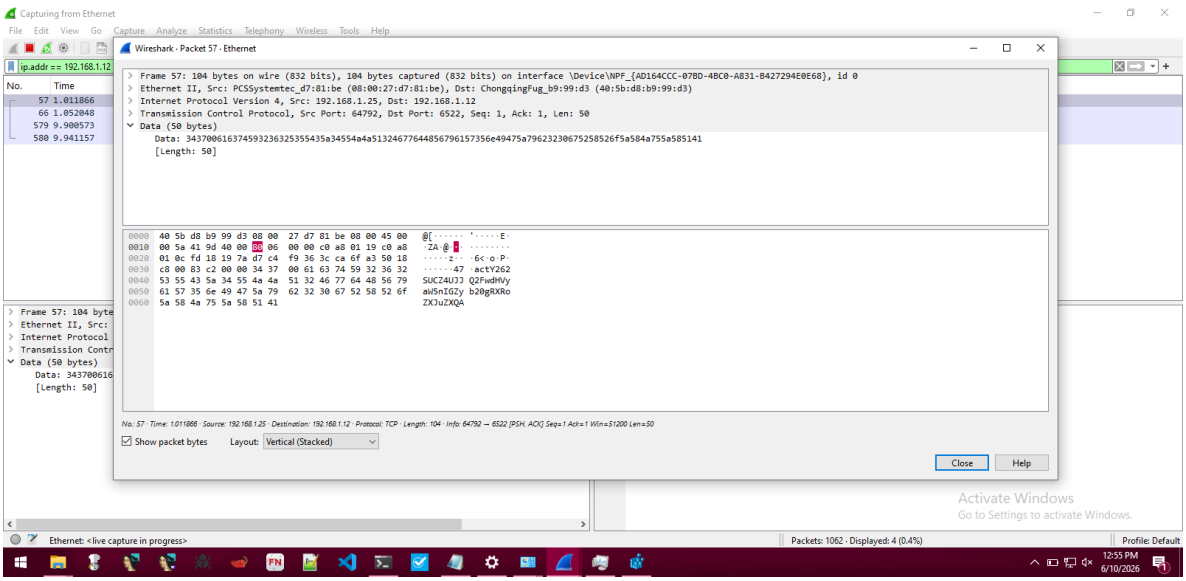

Figure 11 — Beacon packet showing active window title being transmitted to C2

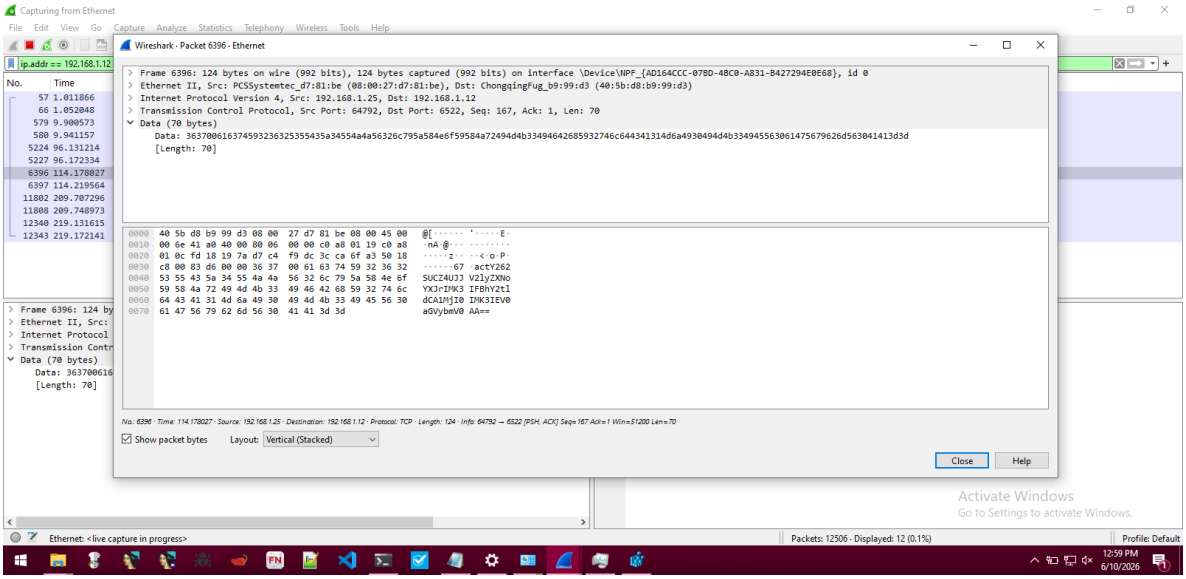

Figure 12 — Second beacon showing active window change (Wireshark capture title visible)

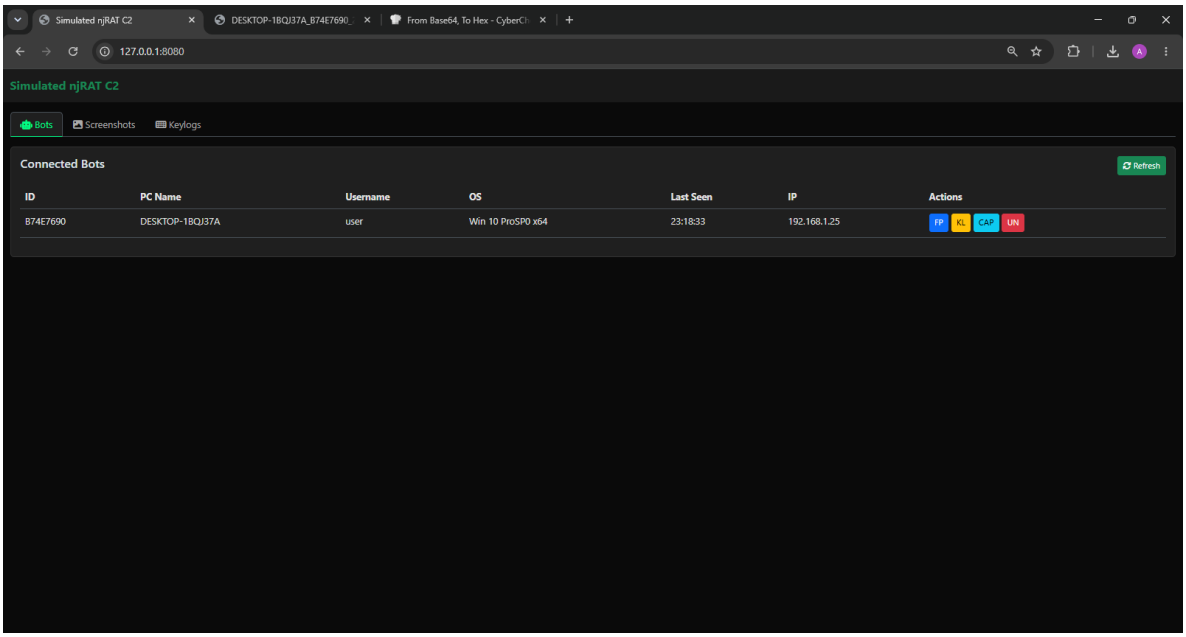

Figure 13 — Simulated C2 panel showing connected bot list with victim fingerprint

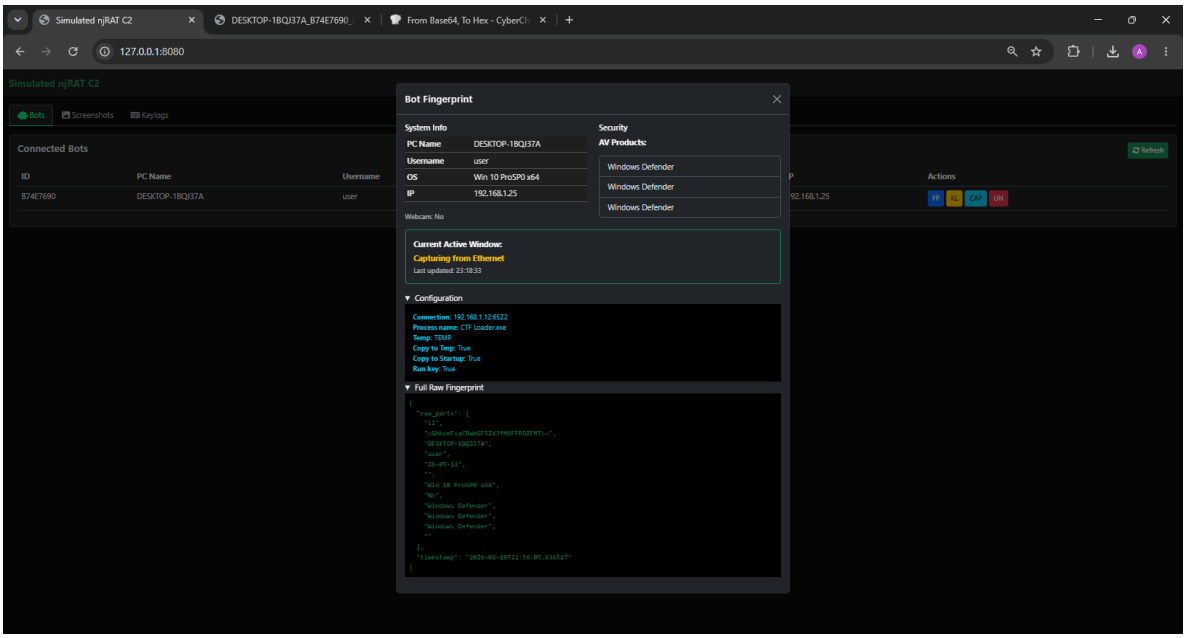

Figure 14 — Full victim fingerprint displayed on C2 panel

9.3 Simulated C2 Session — Keylog Command

The 'kl' command was sent to the implant over the simulated **C2** channel. The implant read the [kl] registry value and returned the cached keystrokes:

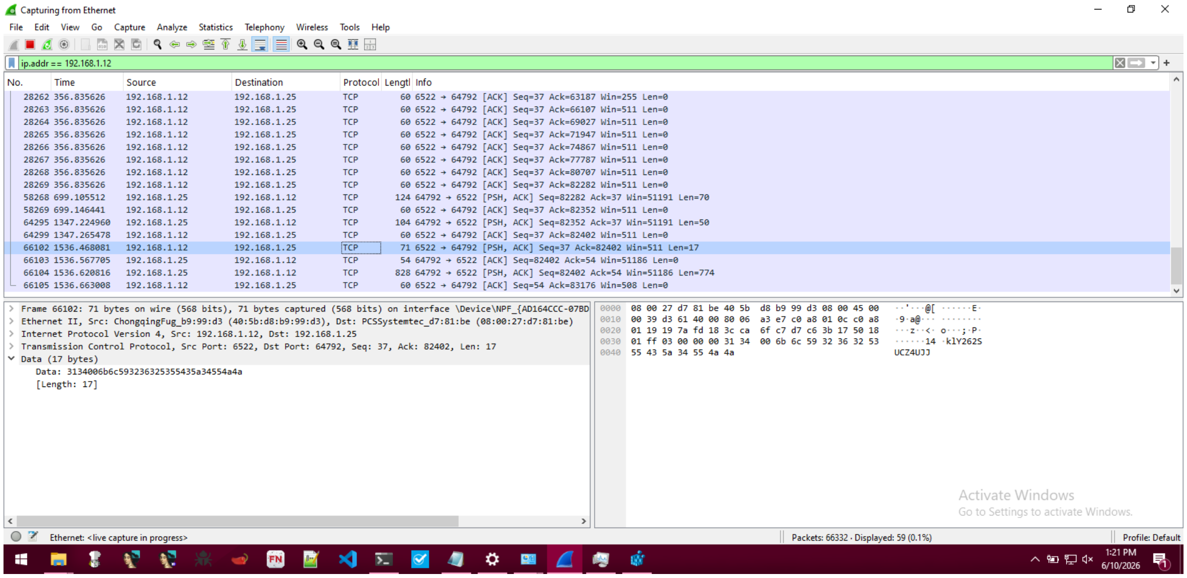

Figure 15 — Sending the 'kl' command from the simulated C2

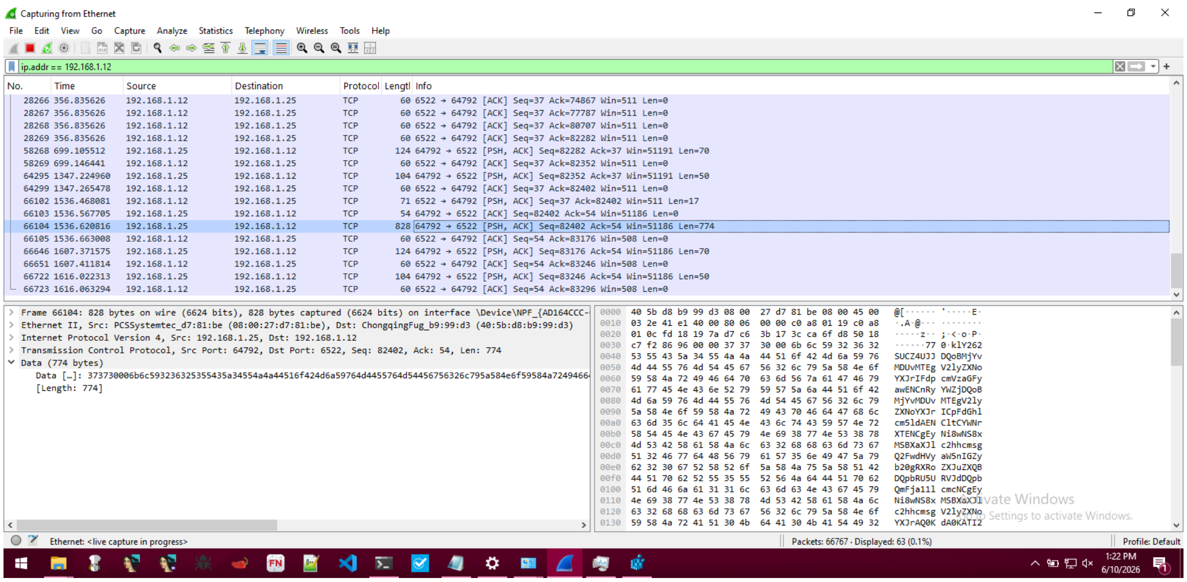

Figure 16 — Implant response containing Base64-encoded keylog data

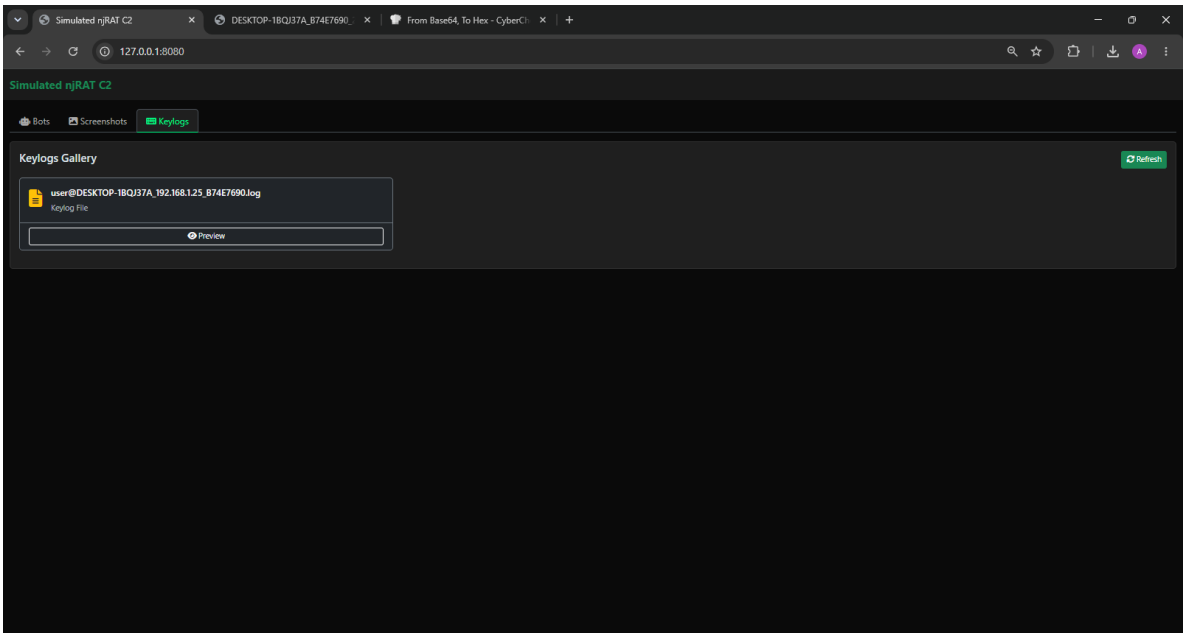

Figure 17 — C2 panel keylog tab receiving dumped keystroke data

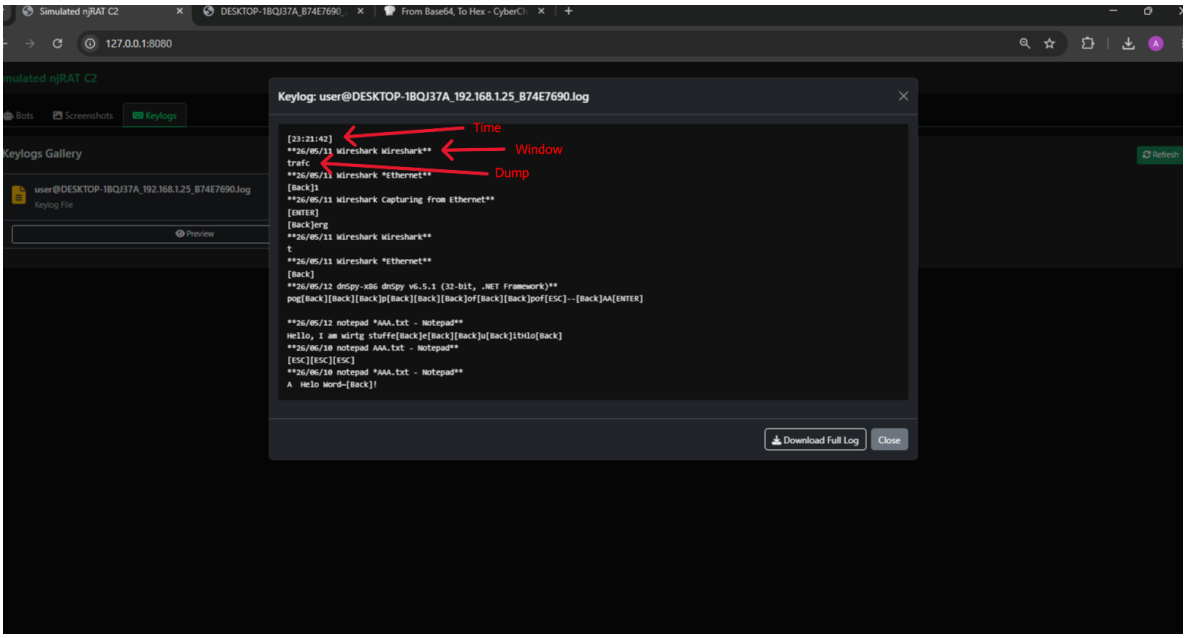

Figure 18 — Annotated keylog dump at the C2 panel

9.4 Simulated C2 Session — Screenshot Command

The 'CAP' command was issued with resolution parameters 800x600. The implant captured the desktop and returned the PNG image data over the **TCP** connection:

```
Command format: 33CAPY262SUCZ4UJJ800Y262SUCZ4UJJ600
```

Annotation Key (response packet dump):
GREEN : Response payload size (bytes)
RED : Command identifier (CAP)
GRAY : Delimiter (Y262SUCZ4UJJ)
BLUE : PNG file header and start of screenshot data

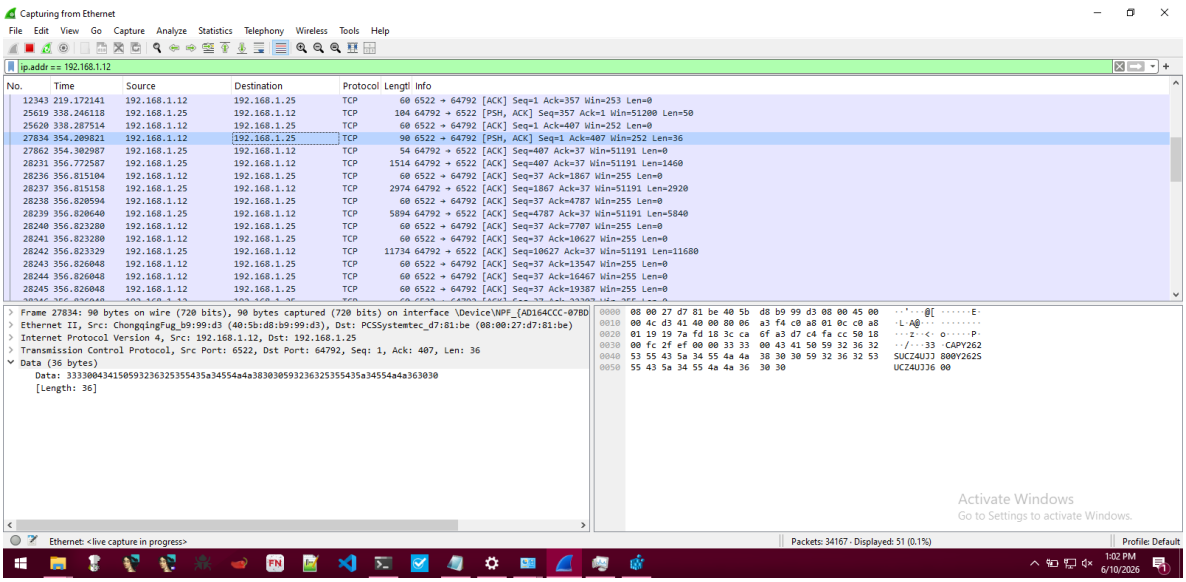

Figure 19 — Issuing the CAP command from the simulated C2

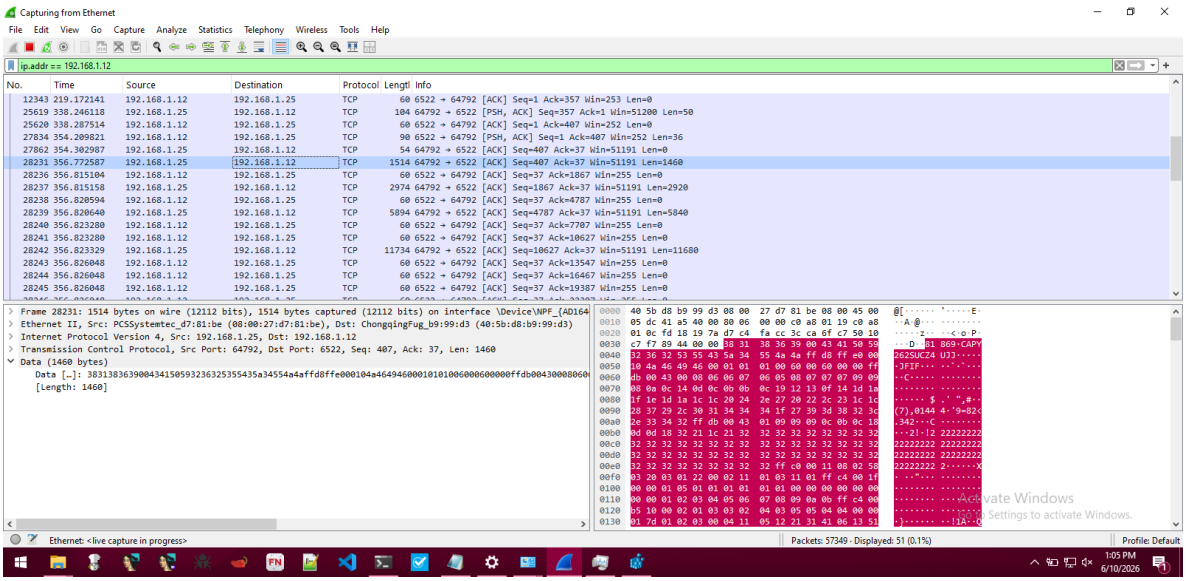

Figure 20 — Raw TCP response from implant containing PNG screenshot data

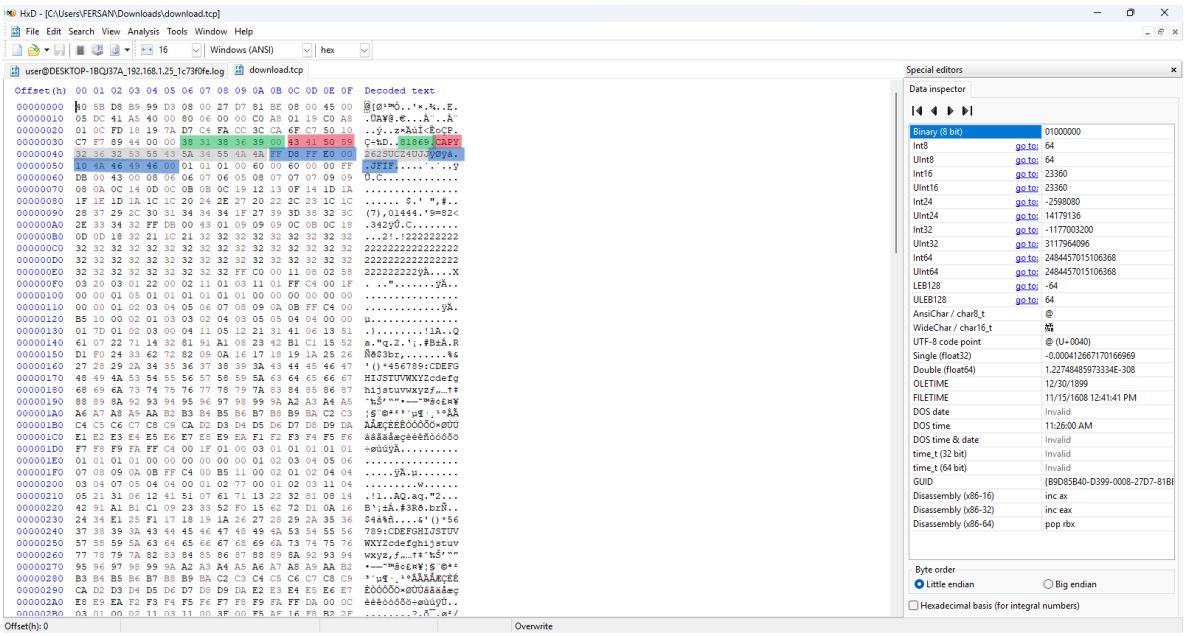

*Figure 21 — **HxD**: Annotated packet dump: size (GREEN), command (RED), delimiter (GRAY), PNG header (BLUE)*

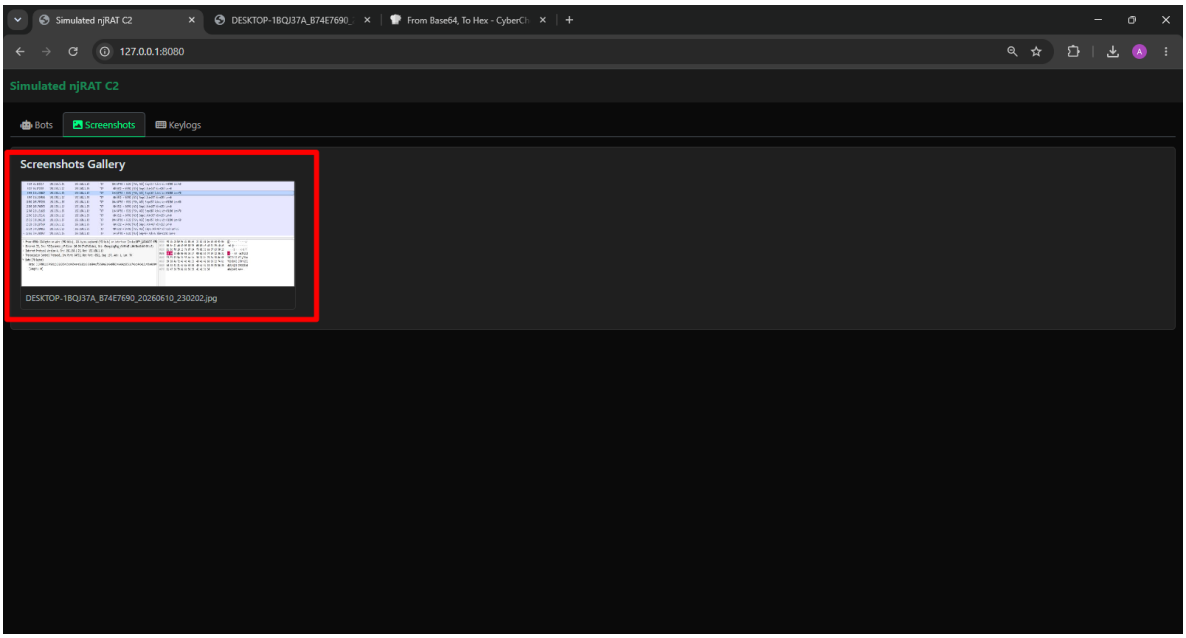

Figure 22 — Annotated C2 panel screenshot tab

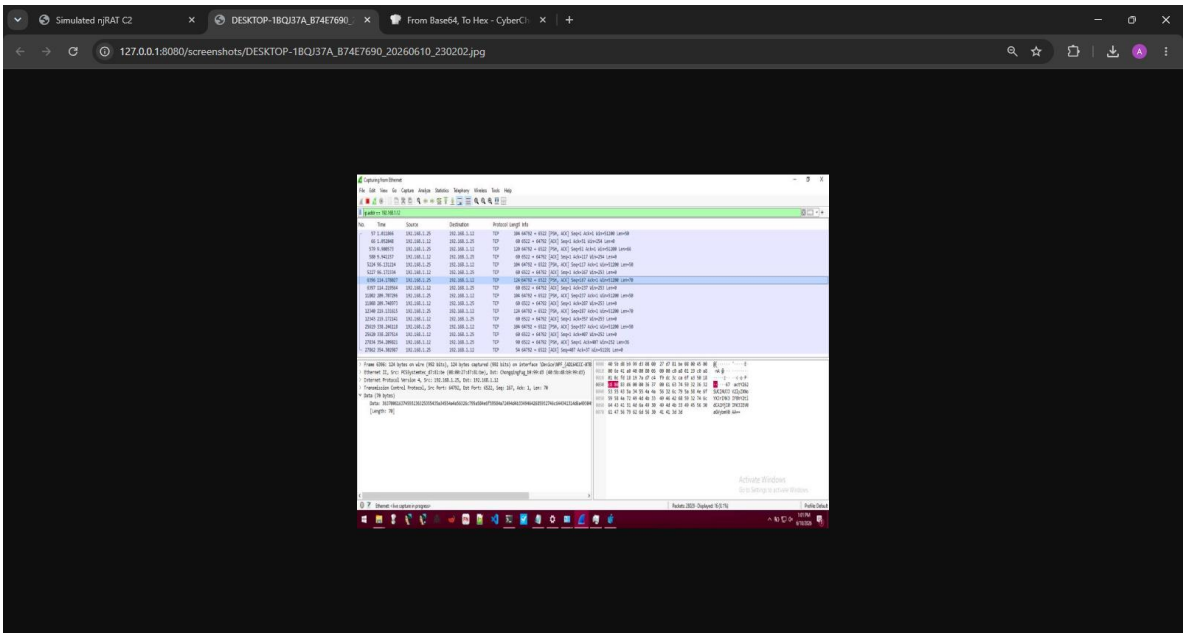

Figure 23 — Rendered screenshot of victim desktop received at C2

10. MITRE ATT&CK Mapping

The following table maps all observed malware behaviors to the MITRE ATT&CK Enterprise framework:

Tactic	Technique	ID	Description
Initial Access	Phishing: Spearphishing via Service	T1566.003	Trojanized CrossFire game binary shared over Discord
Execution	User Execution: Malicious File	T1204.002	Victim runs CTF_Loader.exe / Tools.exe believing it is a game installer
Persistence	Boot/Startup: Registry Run Keys	T1547.001	Adds value under HKCU\Software\Microsoft\Windows\CurrentVersion\Run
Persistence	Boot/Startup: Startup Folder	T1547.001	Drops copy to %APPDATA%\...\Startup\07cc4935...exe
Defense Evasion	Virtualization/Sandbox Evasion	T1497	Checks running processes for 19 analysis tools; exits if found
Defense Evasion	Impair Defenses: Disable FW	T1562.004	Adds firewall allow-rule via netsh.exe for its own binary
Defense Evasion	Hide Artifacts: Zone Identifier	T1553.005	Sets SEE_MASK_NOZONECHECKS=1 to suppress security warnings
Discovery	Process Discovery	T1057	Enumerates running processes and compares them against a blacklist of analysis, monitoring, and security tools before continuing execution.
Discovery	System Owner/User Discovery	T1033	Collects the currently logged-in username during host fingerprinting and transmits it to the C2 server.
Discovery	Software Discovery	T1518	Enumerates installed antivirus/security products and includes the results in victim fingerprint data.
Collection	Screen Capture	T1113	Captures screenshots of the victim desktop via the `CAP` command and transmits the resulting image to the C2 server.

Command and Control	Ingress Tool Transfer	T1105	Receives executable payloads, plugins, and malware updates from the C2 server through the `rn`, `PLG`, and `up` commands.
Execution	Shared Modules	T1129	Dynamically loads arbitrary .NET assemblies (plugins) directly into memory and invokes their methods.
Defense Evasion	Modify Registry	T1112	Creates, modifies, retrieves, and deletes registry values for persistence, configuration storage, and keylog caching.
Defense Evasion	Masquerading	T1036	Uses filenames such as `CTF Loader.exe` to resemble the legitimate Windows CTF Loader component and reduce suspicion.
Credential Access	Input Capture: Keylogging	T1056.001	Hooks keyboard; caches keystrokes in registry under [kl]
Discovery	System Information Discovery	T1082	Fingerprint collects OS, hostname, username, AV, webcam
Collection	Data from Local System	T1005	Retrieves cached keylog buffer from registry via 'kl' C2 command
C2	Non-Application Layer Protocol	T1095	Custom TCP on port 6522 with Base64 + Y262SUCZ4UJJ delimiter
C2	Data Encoding: Standard Encoding	T1132.001	All transmitted data is Base64-encoded before sending
Exfiltration	Exfiltration Over C2 Channel	T1041	Keylogs, screenshots, fingerprint sent to 51.77.192.109:6522
C2	Remote Access Tools	T1219	Full RAT: file exec, plugin load, screenshot, update, uninstall .,etc

11. Indicators of Compromise (IOCs)

11.1 File Hashes

Type	Value
SHA-256	c183d695fa778f9ba71265dbd01cf01936c69ccb4ad0e5c23cad4b242dfca0e9
SHA-1	3ec1e5e14b33b3f389e1e192659764e2f4368de4
MD5	63db40196691060c8b9ec7d7c2292ff3

11.2 File Artifacts

Type	Value
Filename	CTF_Loader.exe
Filename	Tools.exe
Filename	07cc4935e9c1b299f68707836962cbd4.exe
Mutex	07cc4935e9c1b299f68707836962cbd4
Drop Path	%TEMP%\CTF Loader.exe
Drop Path	%APPDATA%\...\Startup\07cc4935e9c1b299f68707836962cbd4.exe

11.3 Network Indicators

Type	Value
C2 IP:Port	51.77.192.109:6522
Protocol	Raw TCP / Custom delimiter protocol
Delimiter	Y262SUCZ4UJJ
Encoding	Base64

11.4 Registry Artifacts

Type	Value
Run Key	HKCU\Software\Microsoft\Windows\CurrentVersion\Run\07cc4935e9c1b299f68707836962cbd4
Keylog Cache	HKCU\SOFTWARE\07cc4935e9c1b299f68707836962cbd4 [kl]
Malware Key	HKCU\SOFTWARE\07cc4935e9c1b299f68707836962cbd4

12. Splunk Detection Queries (SPL)

The following **SPL** queries are designed to detect behaviors exhibited by this **njRAT** variant across Sysmon, Windows Event Logs, network telemetry, and **EDR** data sources. Each query targets a specific **TTP** mapped to the **MITRE ATT&CK** framework.

12.1 Detect Malicious Process Execution (T1204.002)

Detect execution of known **njRAT** dropper filenames by monitoring process creation events:

```
index=sysmon EventCode=1
| where match(lower(Image),
"ctf_loader\.exe|tools\.exe|07cc4935e9c1b299f68707836962cbd4\.exe")
   OR match(lower(CommandLine), "ctf_loader|07cc4935e9c1b299f68707836962cbd4")
| table _time, ComputerName, User, Image, CommandLine, ParentImage,
ParentCommandLine, Hashes
| sort -_time
```

12.2 Detect Registry Run Key Persistence (T1547.001)

Detect writes to HKCU Run key from unexpected processes — a core persistence mechanism of this implant:

```
index=sysmon EventCode=13
| where match(TargetObject,
      "HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run")
| where NOT match(lower(Image),
      "msedge|chrome|firefox|onedrive|teams|office|windowsapps")
| table _time, ComputerName, User, TargetObject, Details, Image, ProcessId
| sort -_time
```

12.3 Detect Startup Folder Binary Drop (T1547.001)

Detect file creation events in the Startup folder — the secondary persistence mechanism:

```
index=sysmon EventCode=11
| where match(lower(TargetFilename),
      "start menu\\programs\\startup\\")
| where match(lower(TargetFilename), "\.exe|\.bat|\.vbs|\.ps1|\.lnk")
| table _time, ComputerName, User, TargetFilename, Image, ProcessId
| sort -_time
```

12.4 Detect Firewall Rule Added via netsh.exe (T1562.004)

Detect the malware's use of netsh.exe to whitelist itself in Windows Firewall:

```
index=sysmon EventCode=1
| where match(lower(Image), "netsh\.exe")
| where match(lower(CommandLine), "firewall") AND match(lower(CommandLine), "add")
   AND match(lower(CommandLine), "allowedprogram|allow")
| table _time, ComputerName, User, Image, CommandLine, ParentImage,
ParentCommandLine
| sort -_time
```

12.5 Detect Zone Identifier Bypass via Environment Variable (T1553.005)

Detect setting of SEE_MASK_NOZONECHECKS environment variable used to suppress file origin warnings:

```
index=sysmon EventCode=13
| where match(lower(TargetObject), "environment")
   AND match(lower(Details), "see_mask_nozonechecks")
| table _time, ComputerName, User, TargetObject, Details, Image
```

Alternatively, hunt in process creation command lines:

```
index=sysmon EventCode=1
| where match(lower(CommandLine), "see_mask_nozonechecks")
| table _time, ComputerName, User, Image, CommandLine, ParentImage
```

12.6 Detect Keylog Cache Write to Registry (T1056.001)

Detect writes to the specific registry key used by the malware to cache keystrokes before **C2** retrieval:

```
index=sysmon EventCode=13
| where match(lower(TargetObject),
      "hkcu\\software\\07cc4935e9c1b299f68707836962cbd4")
   OR match(lower(Details), "\[kl\]")
| table _time, ComputerName, User, TargetObject, Details, Image, ProcessId
| sort -_time
```

12.7 Detect Outbound C2 Connections to Port 6522 (T1071 / T1041)

Detect outbound **TCP** connections to the **C2** IP and port. Even with a dead **C2**, the connection attempt itself is a high-fidelity indicator:

```
index=sysmon EventCode=3
| where DestinationPort=6522
   OR (DestinationIp="51.77.192.109" AND DestinationPort=6522)
| table _time, ComputerName, User, Image, ProcessId,
        SourceIp, SourcePort, DestinationIp, DestinationPort, Protocol
| sort -_time
```

Broader hunt for any unusual non-HTTP/HTTPS outbound on high ports from user-space processes:

```
index=sysmon EventCode=3
| where NOT (DestinationPort=80 OR DestinationPort=443 OR DestinationPort=53)
| where NOT match(lower(Image), "svchost|lsass|explorer|system")
| eval anomaly=if(DestinationPort=6522, "HIGH - njRAT C2 Port", "INVESTIGATE")
| table _time, ComputerName, User, Image, DestinationIp, DestinationPort, anomaly
| sort -_time
```

12.8 Detect Base64-Encoded C2 Beacon in Network Traffic

If you have network packet capture or proxy logs, hunt for the **njRAT** delimiter in raw payload data:

```
index=network sourcetype=stream:tcp dest_port=6522
| where match(payload, "Y262SUCZ4UJJ")
| eval decoded_payload=base64decode(replace(payload, "Y262SUCZ4UJJ", "|"))
| table _time, src_ip, dest_ip, dest_port, payload, decoded_payload
| sort -_time
```

12.9 Detect njRAT Mutex Creation (T1106)

Hunt for creation of the specific **mutex** used by this implant to avoid duplicate execution:

```
index=windows EventCode=4656 OR EventCode=4663
| where match(ObjectName, "07cc4935e9c1b299f68707836962cbd4")
| table _time, ComputerName, SubjectUserName, ObjectName, ProcessName
```

12.10 Broad njRAT Behavioral Hunt — Correlated Alert

High-confidence correlated query that fires when multiple suspicious behaviors are observed from the same process within a time window:

```
| union
    [search index=sysmon EventCode=1
     | where match(lower(Image), "temp|startup|appdata")
       AND match(lower(CommandLine), "07cc4935|ctf_loader|tools\.exe")
     | eval indicator="suspicious_exec"],

    [search index=sysmon EventCode=13
     | where match(lower(TargetObject), "currentversion\\run")
     | eval indicator="regrun_persistence"],

    [search index=sysmon EventCode=3
     | where DestinationPort=6522
     | eval indicator="c2_connection"],
```

```
    [search index=sysmon EventCode=11
     | where match(lower(TargetFilename), "startup\\")
     | eval indicator="startup_drop"]

| stats values(indicator) as indicators, count by ComputerName, User
| where mvcount(indicators) >= 2
| eval risk_score=mvcount(indicators)*25
| table ComputerName, User, indicators, risk_score
| sort -risk_score
```

13. Recommendations

The following recommendations are organized by priority and function. They address immediate containment actions, detection hardening, endpoint policy improvements, user awareness, and long-term architectural improvements.

13.1 Immediate Containment & Eradication

- Isolate all hosts where CTF_Loader.exe, Tools.exe, or 07cc4935e9c1b299f68707836962cbd4.exe have been executed or are present in filesystem telemetry.
- Block the C2 IP 51.77.192.109 at the perimeter firewall and all internal network choke points. Even though the C2 is currently dead, the IP should be permanently blacklisted in case the attacker brings it back online.
- Block outbound TCP on port 6522 at the firewall for all non-server hosts. This port has no legitimate business use for workstations.
- Remove all persistence artifacts on affected hosts: delete the Run key value 07cc4935e9c1b299f68707836962cbd4 from HKCU\Software\Microsoft\Windows\CurrentVersion\Run, remove the Startup folder binary, and delete all copies of the implant from %TEMP% and drive roots.
- Delete the malware's entire registry hive: HKCU\SOFTWARE\07cc4935e9c1b299f68707836962cbd4, including the [kl] keylog cache.
- Remove the Windows Firewall allow-rule created by the malware using: netsh firewall delete allowedprogram <path_to_implant>.
- Revoke and rotate all credentials (passwords, tokens, keys) that may have been entered on the affected host during the infection window — the keylogger may have captured them.
- Assume any active sessions (VPN, web apps, email, banking) authenticated from the infected host are compromised. Force session invalidation across all relevant platforms.

13.2 Network Security Controls

- Implement strict egress filtering at the firewall: deny all outbound traffic on non-standard ports (anything outside 80, 443, 53, 25, 587) from workstations unless explicitly whitelisted.
- Deploy a DNS sinkhole or RPZ (Response Policy Zone) for known malicious domains. The expired domain cf-pharaoh.fun should be added to block lists in case it is re-registered and used in future campaigns.
- Enable deep packet inspection (DPI) on your next-gen firewall to detect Base64-encoded payloads over raw TCP connections — a behavioral signature of this and similar RAT families.
- Deploy network-based intrusion detection (Snort/Suricata) rules specifically for the Y262SUCZ4UJJ delimiter pattern in TCP payloads on any port.
- Implement network segmentation: workstations should not be able to initiate direct outbound connections to the internet. All traffic should pass through an authenticated web proxy with TLS inspection.
- Enable NetFlow or IPFIX logging on all perimeter and internal routers. Unexplained TCP connections to foreign IPs on high ports from workstations are a critical signal for RAT communication.
- Consider deploying a network deception solution (honeypots) that mimics attractive targets. Lateral movement from an infected host will trigger these sensors and provide early warning.

13.3 Endpoint Detection & Response (EDR)

- Deploy a commercial EDR solution (e.g., CrowdStrike Falcon, Microsoft Defender for Endpoint, SentinelOne) with behavioral detection enabled. Signature-only AV is insufficient against compiled .NET RATs with no malicious file content at rest.

- Enable Sysmon with a comprehensive configuration (e.g., SwiftOnSecurity or Olaf Hartong's Sysmon Modular config). At minimum, log: EventCode 1 (process creation), 3 (network connection), 11 (file creation), 13 (registry value set), 17/18 (pipe events).
- Configure EDR behavioral rules to alert on: any process writing to both a Run key AND the Startup folder within the same session, netsh.exe invoked by any non-SYSTEM process, and executable files created under %TEMP% that subsequently establish outbound network connections.
- Enable AMSI (Antimalware Scan Interface) integration in your EDR to inspect .NET assemblies loaded in-memory, including the PLG (plugin) command payloads used by this RAT.
- Enable memory scanning in your EDR to detect reflectively loaded .NET modules (the PLG command injects .NET blobs directly into memory, bypassing disk-based AV scanning).
- Implement application whitelisting (e.g., AppLocker or Windows Defender Application Control) to block execution of any unsigned binary from %TEMP%, %APPDATA%, and removable drive paths — all locations used by this implant.

13.4 Identity & Credential Protection

- Enforce the use of a password manager to ensure users have unique, strong passwords for every account. If the keylogger was active during login sessions, attacker-captured passwords are known bad and must all be rotated.
- Mandate multi-factor authentication (MFA) on all external-facing services (email, VPN, cloud portals, development platforms). Even with captured credentials, the attacker cannot authenticate without the second factor.
- Implement Privileged Access Workstations (PAWs) — dedicated, hardened machines for administrative tasks. Administrative credentials should never be used from general-purpose workstations that may be compromised.
- Audit all accounts that were logged in on infected hosts during the infection window. Check for any new account creation, permission escalation, or unusual login activity in AD event logs (EventCode 4624, 4720, 4732).
- Consider deploying a credential monitoring service (e.g., SpyCloud, HaveIBeenPwned Enterprise) that alerts when corporate credentials appear in breach databases — a downstream consequence of successful keylogging.

13.5 User Awareness & Security Culture

- Launch an urgent targeted awareness communication to all employees explaining that game software, tools, and utilities shared via Discord, Telegram, or personal messaging platforms must NEVER be executed on corporate or personal machines used for work. The threat vector in this campaign was social trust within a gaming community.
- Conduct phishing and social engineering simulation exercises specifically covering the 'trojanized game / cracked software' scenario. Many users do not associate game downloads with malware risk.
- Establish a clear, frictionless process for employees to report suspicious files or processes without fear of blame. The victim in this case experienced significant system slowdown — they needed an easy path to report this before further damage occurred.
- Educate users on the WoW64 indicator: while highly technical, simplified awareness messaging (e.g., 'if a program claims to be a game but runs as a 32-bit process on a 64-bit machine, report it immediately') can be effective for technical teams.
- Include 'Discord and gaming platform threats' as a dedicated topic in annual security awareness training — these platforms are increasingly weaponized for malware distribution targeting younger employees and gamers.

13.6 Hardening Windows Baseline

- Disable the SEE_MASK_NOZONECHECKS environment variable at the GPO level. Any process attempting to set this variable should trigger an alert. This flag is virtually never needed in a corporate environment.
- Configure Windows Defender Attack Surface Reduction (ASR) rules, specifically: 'Block executable files from running unless they meet a prevalence, age, or trusted list criterion', 'Block process creations originating from PSExec and WMI commands', and 'Block untrusted and unsigned processes that run from USB'.
- Enable Controlled Folder Access in Windows Defender to protect the Startup folder and %APPDATA% from unauthorized writes by non-whitelisted processes.
- Restrict netsh.exe execution via AppLocker or WDAC policies to SYSTEM and whitelisted administrator accounts only. User-space processes should never need to invoke netsh.exe in a well-managed environment.
- Disable AutoRun and AutoPlay on all removable media to prevent the Tools.exe drop-to-all-drives technique from propagating the implant.
- Enable Windows event logging for: Registry Audit (EventCode 4657), Process Creation Audit with command line (EventCode 4688 + ProcessCreationIncludeCmdLine=1), Object Access Audit for Startup folder and Run key paths.
- Ensure Windows Defender real-time protection and cloud-delivered protection are enabled and not disabled by GPO or registry overrides. This malware uses netsh.exe to bypass host firewall — verify that host firewall enforcement cannot be bypassed by standard user accounts.

13.7 Threat Intelligence & Ongoing Hunting

- Add all IOCs from Section 10 to your threat intelligence platform (MISP, OpenCTI, Anomali ThreatStream) and push them to all enforcement points (SIEM, firewall, EDR, email gateway, proxy).
- Subscribe to threat feeds that track njRAT variants and commodity RAT campaigns. This family is widely used and new variants are released frequently by multiple threat actors.
- Schedule recurring Splunk searches (see Section 11) as saved alert searches with a frequency of at least every 15 minutes for the highest-fidelity queries (C2 port detection, Run key writes, startup folder drops).
- Establish a threat hunting cadence: run the correlated behavioral hunt query (Section 11.11) weekly across your entire estate to catch any hosts that may have been infected but not yet triggered automated alerts.
- Monitor the Discord, Telegram, and TikTok presence associated with this campaign for reactivation. The actor has a history of using social media for distribution. Although the C2 and domain are currently offline, the actor may resume activity.
- Implement a process for tracking 'low and slow' C2 behavior: this RAT beacons the active window title continuously. Set up SIEM correlations to detect a single host making repeated outbound TCP connections to the same foreign IP over multiple days — even at low frequency.

13.8 Incident Response Preparedness

- Ensure your IR playbook includes a specific procedure for RAT/keylogger incidents that covers: mandatory credential rotation scope, session revocation steps, timeline reconstruction for keylog activity, and legal/HR notification thresholds.
- Maintain offline, immutable backups of all critical systems. This RAT includes an 'rn' command (run PE) that could be used to deploy ransomware as a second-stage payload.
- Establish a formal malware analysis capability (sandboxed VM + Flare-VM) so future samples can be analyzed internally within hours rather than days. The speed of analysis directly impacts containment effectiveness.
- Conduct a post-incident tabletop exercise simulating a scenario where this RAT had been active for 30 days before detection. This exercise will surface gaps in your detection coverage and help prioritize the recommendations above.

Appendix A — Glossary

Type	Value
RAT	Remote Access Trojan — malware giving attacker full remote control of victim machine
njRAT	A commodity .NET RAT originally developed in the Middle East region; widely distributed
CTF Loader	Legitimate Windows component associated with the Microsoft Text Services Framework. The genuine process is typically ctfmon.exe and is responsible for managing language input services, speech recognition, handwriting recognition, keyboard layouts, and other alternative user input functions. It normally executes from trusted Windows system directories. Typically it runs as x64 process under x64 Windows systems.
C2	Command & Control — attacker-controlled server that issues commands to infected hosts
FakeNet-NG	Network simulation tool that intercepts all outbound connections for analysis
WoW64	Windows 32-bit on Windows 64-bit emulation layer; 32-bit processes run in SysWOW64
Mutex	Mutual exclusion object; used by malware to prevent duplicate infection
SPL	Splunk Processing Language — query language for Splunk SIEM
MITRE ATT&CK	Globally accessible knowledge base of adversary tactics and techniques
IOC	Indicator of Compromise — forensic artifact indicating a host may be compromised
ASR	Attack Surface Reduction — Windows Defender rules blocking common attack patterns
AMSI	Antimalware Scan Interface — Windows API for runtime inspection of scripts/assemblies
TLP	Traffic Light Protocol — information sharing classification (AMBER = restricted)
dnSpy	Open-source .NET reverse-engineering and debugging tool used to decompile, inspect, modify, and recompile .NET assemblies. Used in this analysis to review source code, identify malware functionality, and patch anti-analysis mechanisms.
Wireshark	Network protocol analyzer used to capture, inspect, and decode network traffic. Used in this analysis to monitor communications between the malware and the simulated C2 server and to examine protocol messages and exfiltrated data.
HxD	Hex editor used to inspect and analyze binary files and raw data at the byte level. Used in this analysis to examine packet contents, verify protocol structures, and identify file signatures such as PNG screenshot data.
Flare-VM	Windows-based malware analysis environment maintained by Mandiant that provides a curated collection of reverse-engineering, forensic, and incident response tools. Used as the primary analysis workstation for this investigation.
Bootstrap	Front-end CSS framework used to develop responsive web interfaces. Used in the custom simulated C2 web panel developed for this analysis.
Vanilla JavaScript	JavaScript implemented without external frameworks or libraries. Used in the simulated C2 frontend to handle user interaction and communication with the backend.